SAÉ S5 parcours DATAIA

BUT Informatique de Reims

Lydie Sandron, Ingrid Zmölnig, Frédéric Blanchard

Année universitaire 2026/2027

VERSION DE TRAVAIL

Résumé

La SAÉ du semestre 5 est un projet réalisé en groupes de trois étudiants de troisième année de BUT Informatique de Reims, en formation initiale, parcours DATAIA. Le projet représente cinq semaines de travail à temps plein, réparties en trois blocs entre le 21 septembre et le 27 novembre 2026, et se termine par une soutenance devant un jury et par la remise d'une vidéo de démonstration en anglais. Chaque groupe choisit l'un des huit sujets suivants, en accord avec l'équipe enseignante.

- Sujet 1 : société d'agents et étude des comportements
- Sujet 2 : datawarehouse en langage naturel contre solution classique
- Sujet 3 : évaluation de harnais d'agents pour le développement logiciel
- Sujet 3bis : harnais d'agent pour la data science
- Sujet 3ter : harnais d'agent pédagogique pour étudiants en informatique
- Sujet 4 : transformeurs pour données tabulaires
- Sujet 5 : « humain ou IA ? » sur un serveur Minecraft
- Sujet 6 : monitoring d'un JupyterHub et prédiction des surcharges, crashs et redémarrages

iut REIMS | CHÂLONS CHARLEVILLE INSTITUT UNIVERSITAIRE DE TECHNOLOGIE

UNIVERSITÉ DE REIMS CHAMPAGNE-ARDENNE

Table des matières

1 Cadre général et organisationnel **3**
1.1 Public et format . . . 3
1.2 Calendrier . . . 3
1.3 Ressources et contraintes . . . 4
1.4 Dépôt et journal de bord . . . 4
1.5 Suivi et évaluation . . . 5

2 Les sujets **6**
Sujet 1 : société d'agents et étude des comportements . . . 6
Sujet 2 : datawarehouse en langage naturel contre solution classique . . . 6
Sujet 3 : évaluation de harnais d'agents pour le développement logiciel . . . 7
Sujet 3bis : harnais d'agent pour la data science . . . 8
Sujet 3ter : harnais d'agent pédagogique pour étudiants en informatique . . . 9
Sujet 4 : transformeurs pour données tabulaires . . . 10
Sujet 5 : « humain ou IA ? » sur un serveur Minecraft . . . 11
Sujet 6 : monitoring d'un JupyterHub et prédiction des surcharges, crashs et redémarrages 12

VERSION DE TRAVAIL

1 Cadre général et organisationnel

1.1 Public et format

La SAÉ est proposée aux étudiants de troisième année de BUT Informatique de Reims, en formation initiale, parcours DATAIA. Elle se déroule au semestre 5, en groupes de trois étudiants. Chaque groupe traite un sujet, choisi en accord avec l'équipe enseignante.

Six sujets utilisent un grand modèle de langage (LLM). Les sujets 1, 3, 3bis, 3ter et 5 construisent ou évaluent des agents. Le sujet 2 utilise un LLM pour interroger une base de données. Les sujets 4 et 6 n'en utilisent pas : le sujet 4 compare des modèles d'apprentissage automatique, le sujet 6 supervise un serveur et prédit ses incidents.

Un agent est un LLM qui réalise une tâche en plusieurs étapes en utilisant des outils. Un harnais d'agent est le programme qui entoure le LLM : il lui donne accès à des outils (lecture de fichiers, exécution de code, recherche), répète la boucle « le modèle propose une action, le programme l'exécute et lui renvoie le résultat », et contrôle ce que le modèle a le droit de faire.

Chaque sujet est décrit selon trois dimensions. Les données : ce que le projet produit ou utilise, et comment ces données sont constituées et contrôlées. L'observabilité : ce que le système enregistre pour que son fonctionnement et ses résultats puissent être vérifiés après coup. L'architecture : le découpage du système en composants et son déploiement. Chaque groupe documente aussi les limites de son approche, autant que ses résultats.

1.2 Calendrier

Cinq semaines de travail à temps plein, du 21 septembre au 27 novembre 2026, réparties en trois blocs et entrecoupées d'autres enseignements.

- **Semaines 39 et 40** (21 septembre au 4 octobre) : cadrage du sujet, choix d'architecture, mise en place de l'infrastructure, vérification des sorties structurées (voir ci-dessous), première version fonctionnelle. La note technique intermédiaire est rédigée avant l'interruption du projet.
- **Semaine 43** (19 au 25 octobre) : restitution collective en début de semaine, puis poursuite du développement.
- **Semaines 47 et 48** (16 au 29 novembre) : reprise, consolidation, expérimentations, note technique finale, vidéo de démonstration en anglais. Soutenance le vendredi 27 novembre devant un jury.

VERSION DE TRAVAIL

1.3 Ressources et contraintes

- **Machines virtuelles.** Chaque groupe déploie son projet sur des machines virtuelles fournies par le département (OpenNebula). Ces machines sont partagées entre les groupes : le nombre, la taille et la durée de vie des machines utilisées sont à justifier.
- **Passerelle LiteLLM.** L'accès aux LLM passe par un seul point d'entrée, la passerelle LiteLLM, qui redirige les requêtes vers un serveur de modèles (vLLM) hébergé sur le calculateur de l'université. Tous les groupes la partagent : le système gère donc les erreurs et les délais de réponse, et limite le nombre de requêtes qu'il envoie.
- **Vérification en semaine 1 (sujets 1, 2, 3, 3bis, 3ter et 5).** Le modèle doit savoir appeler des fonctions (`function calling`) et répondre dans un format imposé, par exemple JSON (sorties structurées, ou `structured output`). La plupart des outils cités en dépendent. Si le modèle ne le permet pas, ou le fait de façon instable, une solution de repli est documentée : redemander une réponse quand le format est incorrect, ou changer de modèle.
- **Isolation du code généré (sujets 3 et 3bis).** Un conteneur partage le noyau de la machine hôte. Une micro-VM, machine virtuelle très légère qui démarre en une fraction de seconde, a son propre noyau et isole donc davantage ; il en existe aujourd'hui de nombreuses implémentations libres. Les micro-VM demandent la virtualisation matérielle (KVM) : sa disponibilité à l'intérieur des machines virtuelles du département (virtualisation imbriquée) est à vérifier en semaine 1.
- **Socle commun.** Python, déploiement dans des conteneurs, dépôt sur le GitLab de l'IUT. Kubernetes est exclu : trop lourd pour ce projet. Pour le sujet 5, le serveur de jeu et les bots peuvent utiliser Java ou Node.js, à justifier.
- **Sujets 4 et 6.** Ces sujets n'ont pas besoin de la passerelle ni de la vérification ci-dessus, sauf si un agent automatise une partie du travail. Pour le sujet 4, le temps de calcul et la mémoire des machines virtuelles limitent le nombre et la taille des expériences.
- **Technologies proposées.** Celles citées dans chaque sujet sont des suggestions. Le groupe choisit librement et justifie son choix.

1.4 Dépôt et journal de bord

Un dépôt GitLab par groupe, créé dès la semaine 1 sur l'instance de l'IUT.

- **Structure.** Arborescence claire : code, infrastructure, données et documentation dans des dossiers séparés. Aucun fichier généré, mot de passe ou clé d'accès dans le dépôt ; le `.gitignore` est complété dès le départ.
- **README.md.** Présent à la racine et tenu à jour : objet du projet et question retenue, installation et lancement, organisation du dépôt, lien vers le journal de bord.

VERSION DE TRAVAIL

- **Commits.** Réguliers et de taille raisonnable : un commit correspond à une modification cohérente. Le message indique ce qui change et pourquoi. L'historique reflète le travail réellement effectué et n'est pas reconstitué après coup.
- **Journal de bord.** Fichier `JOURNAL.md` ou wiki GitLab, tenu comme un carnet de laboratoire : une entrée datée par séance de travail, qui note ce qui a été fait, les décisions prises et leurs raisons, les problèmes rencontrés. Il sert de base aux notes techniques et facilite la reprise après chaque interruption du calendrier.

1.5 Suivi et évaluation

- **Réunions hebdomadaires.** D'une à deux heures chaque semaine avec les enseignants. L'évaluation est continue et porte sur la capacité du groupe à identifier seul ses difficultés, la justification de ses choix, la compréhension du projet par chacun de ses membres, et la capacité à revenir sur un échec ou un résultat inattendu.
- **Notes techniques.** Deux par groupe : l'une au point d'étape, l'autre en fin de projet. Chacune traite une ou deux parties du travail dont la solution peut servir dans un autre contexte. Elle expose le problème rencontré, les pistes essayées puis écartées, la solution retenue (assez détaillée pour être reproduite par quelqu'un d'autre), ses limites et son coût (temps passé, requêtes envoyées à la passerelle, complexité ajoutée). Deux pages au maximum, écrites pour un lecteur extérieur au groupe. Elle sert aussi de support à la restitution orale.
- **Relevé de temps.** Chaque groupe relève son temps de travail par type d'activité (cadrage, développement, infrastructure, expérimentation, rédaction, réunions) avec Kimai, TimeTagger ou un outil équivalent. Ce relevé fournit le coût indiqué dans les notes techniques et alimente la discussion sur la répartition du travail dans le groupe.
- **Restitution collective.** En semaine 43, chaque groupe présente sa note technique intermédiaire aux autres groupes, qui posent des questions et font des remarques.
- **Vidéo de démonstration.** Chaque groupe produit une vidéo de démonstration de son projet, en anglais. Elle est évaluée par l'enseignante d'anglais, qui en précise les modalités.
- **Évaluation.** Elle porte autant sur la démarche que sur le résultat : chaque choix doit pouvoir être justifié, y compris ceux mis en œuvre par un autre membre du groupe ; les cas d'échec sont analysés ; la note technique est compréhensible par un lecteur extérieur au groupe. Une checklist de préparation est fournie séparément.
- **IA générative.** L'usage d'assistants d'IA générative pour produire du code ou de la rédaction est autorisé. Chaque élément du projet doit pouvoir être expliqué, ligne par ligne si besoin, par n'importe quel membre du groupe.

VERSION DE TRAVAIL

2 Les sujets

Sujet 1 : société d'agents et étude des comportements

Plateforme de type réseau social restreint, où des agents fondés sur un LLM publient, s'abonnent les uns aux autres et réagissent aux publications. Chaque agent a un profil distinct (centres d'intérêt, ton, opinions). Le sujet étudie ce qui se passe quand ces agents interagissent longtemps entre eux. Le groupe fixe en semaine 1 la question précise à laquelle le projet répond, par exemple : le profil d'un agent change-t-il au fil des interactions ? Une information fausse introduite volontairement se propage-t-elle ? Quel est l'effet d'un agent modérateur sur les échanges ? Les enseignants valident cette question avant le développement, car elle détermine la conception.

- **Données.** Un protocole d'observation défini à l'avance : ce qui est enregistré (publications, abonnements, réactions, horodatage), le nombre d'exécutions, des conditions identiques d'une exécution à l'autre. Une analyse chiffrée répond à la question retenue.
- **Observabilité.** Chaque appel au modèle est enregistré (contenu, nombre de jetons, c'est-à-dire d'unités de texte comptées par le modèle, temps de réponse) avec l'agent qui l'a émis et l'effet obtenu sur la plateforme. Ces traces permettent de retrouver, après coup, pourquoi un agent s'est comporté d'une certaine façon.
- **Architecture.** Composants distincts : interface, service applicatif, coordination des agents, stockage, accès à la passerelle. Le déploiement limite le nombre de machines virtuelles utilisées. Les décisions sont documentées, en particulier l'exécution des agents en parallèle et le comportement du système quand la passerelle est indisponible.
- **Livrables.** Système et code source, protocole et données produites, deux notes techniques, synthèse répondant à la question posée.
- **Limites.** Le coût augmente avec le nombre d'agents et de cycles d'interaction, ce qui limite la taille des expériences. Le modèle ne donne pas toujours la même réponse : deux exécutions identiques ne produisent pas exactement les mêmes résultats.
- **Pistes techniques.** Coordination des agents (orchestration) : CrewAI, LangGraph. Stockage des interactions : PostgreSQL, SQLite. Mémoire des agents, si le groupe leur en donne une : base de données vectorielle Chroma ou Qdrant. Suivi des appels au modèle : Langfuse, Phoenix.

Sujet 2 : datawarehouse en langage naturel contre solution classique

Un entrepôt de données que l'on interroge en langage courant : un LLM traduit la question en requête SQL, et la requête est vérifiée avant d'être exécutée. Cette approche est comparée, sur le

VERSION DE TRAVAIL

même entrepôt, à une solution classique : tableau de bord à filtres prédéfinis, requêtes SQL écrites à la main, ou tout autre accès sans LLM. Le groupe choisit en semaine 1 un domaine de données concret, de préférence des données ouvertes, et le justifie auprès des enseignants.

- **Données.** Schéma de l'entrepôt documenté : tables, granularité (ce que représente une ligne), relations. Sources chargées avec un contrôle de qualité.
- **Observabilité.** La comparaison suit un protocole défini à l'avance : mêmes questions posées aux deux solutions, mesure du taux de réussite, du temps de réponse et du comportement face aux questions ambiguës ou mal formulées. Pour l'approche en langage courant, un journal enregistre chaque requête SQL générée, son sort (exécutée, ou rejetée avec le motif) et son résultat.
- **Architecture.** L'approche en langage courant comporte trois couches séparées : traduction en SQL, validation (respect du schéma, lecture seule), moteur de base de données. La solution classique est développée en parallèle sur le même entrepôt et ne dépend pas de la première.
- **Livrables.** Entrepôt et schéma documenté, les deux solutions d'interrogation, protocole et résultats de la comparaison, deux notes techniques.
- **Limites.** Une requête peut être correcte du point de vue de la syntaxe et du schéma tout en répondant à une autre question que celle posée : ce cas est à détecter et à mesurer. Les conclusions valent pour le domaine de données et les questions testés.
- **Pistes techniques.** Base de données analytique embarquée : DuckDB. Traduction en SQL : Vanna, moteur de requête en langage naturel de LlamaIndex, ou solution construite sur la passerelle. Solution classique : SQL direct, ou outil léger de tableaux de bord.

Sujet 3 : évaluation de harnais d'agents pour le développement logiciel

Le groupe évalue un ou plusieurs harnais d'agents de programmation, c'est-à-dire des agents qui résolvent des tickets, écrivent du code ou corrigent des bugs. L'évaluation porte sur des tâches dont la réussite se vérifie automatiquement, typiquement par des tests unitaires que le code produit doit faire passer. Le groupe ne construit pas de harnais : il compare plusieurs harnais entre eux, ou un harnais à une méthode de référence, comme un simple appel direct au LLM. Il choisit et justifie en semaine 1 les harnais évalués, le langage et le domaine des tâches, ainsi que le protocole de comparaison. Contrainte sans dérogation : le code produit par un harnais évalué n'a pas été relu et peut endommager la machine ; il ne s'exécute donc jamais hors d'un environnement isolé (bac à sable), y compris pendant les expérimentations.

- **Données.** Un jeu de tâches dont la réussite se vérifie automatiquement. Le groupe justifie la variété des tâches (langages, types de tâches, difficulté) et leur origine : benchmark existant

VERSION DE TRAVAIL

repris ou adapté, ou tâches écrites par le groupe.

- **Observabilité.** Chaque tentative est enregistrée : appels d'outils, modification de code proposée (patch), résultat des tests, nombre d'itérations, coût. Les résultats sont réunis dans un tableau que quelqu'un d'autre peut reproduire. Les échecs sont consignés et classés : tâche mal comprise, modification incorrecte, boucle qui ne se termine pas.
- **Architecture.** Le code généré s'exécute uniquement dans un bac à sable, dont le niveau d'isolation (conteneur ou micro-VM) est justifié. Le pipeline d'évaluation peut être relancé à l'identique : le jeu de tâches, la configuration de chaque harnais et les limites par tâche (nombre d'itérations, de jetons, durée) sont versionnés.
- **Livrables.** Pipeline d'évaluation, jeu de tâches et résultats bruts, rapport comparatif, deux notes techniques.
- **Limites.** Les résultats valent pour les tâches et les langages testés : un harnais réglé pour un type de tâche peut être moins bon sur un autre. Le coût des appels au modèle limite le nombre de tâches et de tentatives.
- **Pistes techniques.** Harnais : Pi, OpenHands, Aider, SWE-agent, ou harnais minimal construit sur la passerelle. Tâches : un sous-ensemble de SWE-bench (tickets réels de projets open source, avec leurs tests) ou des tâches propres au groupe avec tests automatiques. Isolation : conteneurs (Docker Compose, Podman) ou micro-VM (Firecracker, Kata Containers, Cloud Hypervisor, E2B). Suivi des expériences : Langfuse, Phoenix.

Sujet 3bis : harnais d'agent pour la data science

Le sujet 3 évalue des harnais existants ; le sujet 3bis en construit un, spécialisé dans l'analyse de données. À partir d'un jeu de données et d'une question posée en langage courant, l'agent explore les données, exécute des analyses (statistiques descriptives, graphiques, tests), construit et évalue un modèle si nécessaire, puis rédige un rapport. Le groupe choisit et justifie en semaine 1 un ou plusieurs jeux de données ouverts et quelques scénarios d'usage représentatifs (une question posée à l'agent et la réponse attendue), sur lesquels le harnais est évalué. Contrainte sans dérogation : le code généré par l'agent ne s'exécute jamais hors d'un environnement isolé, y compris pendant le développement.

- **Données.** Jeux de données ouverts couvrant plusieurs types d'analyse (exploration, statistiques, modélisation). Chaque scénario a un résultat de référence, qui permet de juger ce que produit l'agent.
- **Observabilité.** Chaque étape de l'agent est enregistrée : code généré, code exécuté, résultat obtenu, erreur éventuelle. Ces traces montrent si les conclusions reposent sur une exécution réelle ou sur une réponse plausible mais inventée par le modèle.

VERSION DE TRAVAIL

- **Architecture.** L'exécution du code se fait dans un bac à sable, dont le niveau d'isolation (conteneur ou micro-VM) est justifié, séparé de la boucle de l'agent et de ses outils. Les décisions sur l'accès aux données et sur la présentation des résultats (texte, figures, tableaux) sont documentées.
- **Livrables.** Harnais et code, jeux de données et scénarios d'évaluation, rapport d'évaluation du harnais sur ces scénarios, deux notes techniques.
- **Limites.** La fiabilité dépend de la justesse du code écrit par le modèle. Une erreur subtile (mauvais filtre, échantillon biaisé) ne provoque aucune erreur d'exécution et peut mener à une conclusion fausse. Les conclusions valent pour les jeux de données et les scénarios testés.
- **Pistes techniques.** Base du harnais : Pi étendu, Open Interpreter, PandasAI, ou harnais minimal construit sur la passerelle. Exécution du code : noyau Jupyter isolé. Isolation : conteneurs (Docker Compose, Podman) ou micro-VM (Firecracker, Kata Containers, Cloud Hypervisor, E2B). Suivi des appels au modèle : Langfuse, Phoenix.

Sujet 3ter : harnais d'agent pédagogique pour étudiants en informatique

Harnais d'agent destiné aux étudiants du BUT Informatique de Reims : un assistant qui aide à comprendre et à apprendre, sans faire le travail à la place de l'étudiant. Trois éléments sont à construire. Des garde-fous, c'est-à-dire des règles qui distinguent une demande d'aide légitime (comprendre une erreur, clarifier une notion) d'une demande de solution complète à un travail évalué, et qui réagissent différemment : donner un indice, poser une question en retour, ou refuser en expliquant pourquoi. Des skills pédagogiques, c'est-à-dire des ensembles d'instructions que l'agent active selon le contexte : expliquer un concept sans donner le code, relire un code en posant des questions sur les choix faits, proposer un exercice voisin mais différent. Des serveurs MCP (Model Context Protocol, protocole standard qui permet à un agent d'accéder à des sources de données) qui donnent à l'agent accès à des ressources du département : contenus de cours, planning, dépôt GitLab de l'étudiant.

Contrainte sans dérogation : les serveurs MCP ne donnent accès qu'à des ressources publiques ou propres à l'utilisateur qui interroge le harnais, jamais au travail d'un autre étudiant sans son accord. Le harnais reste un prototype testé par le groupe et par des volontaires ; il n'est pas déployé auprès de l'ensemble du département.

- **Données.** Un jeu de demandes types, classées en catégories définies à l'avance (aide légitime, demande de solution complète, cas ambigus), sert à évaluer les garde-fous. Les ressources fournies via MCP sont décrites : nature, provenance, date de dernière mise à jour.
- **Observabilité.** Chaque interaction est enregistrée : skill activée, garde-fou déclenché ou

VERSION DE TRAVAIL

non et pourquoi, ressource MCP consultée. Le taux de détection est mesuré sur le jeu de demandes types. Les erreurs sont classées : aide légitime bloquée à tort, ou solution complète donnée alors qu'elle aurait dû être refusée.

- **Architecture.** La boucle de l'agent, les skills, les garde-fous (appliqués avant ou après la réponse du modèle) et les serveurs MCP sont des composants séparés, testables chacun isolément.
- **Livrables.** Harnais et code, jeu de demandes types et catégories, mesure de l'efficacité des garde-fous, serveurs MCP documentés, deux notes techniques.
- **Limites.** Un garde-fou qui résiste aux contournements testés ne garantit rien face à des formulations imprévues ; le rapport le précise. Les ressources fournies via MCP reflètent l'état du département à un instant donné et se mettent à jour à la main.
- **Pistes techniques.** Base du harnais : Pi étendu, ou harnais minimal construit sur la passerelle. Garde-fous : classification de chaque demande par un appel distinct au modèle (réponse dans un format imposé), règles écrites à la main, ou les deux. Serveurs MCP : kit de développement officiel, au-dessus des API existantes du département (par exemple celle de GitLab). Suivi des appels au modèle : Langfuse, Phoenix.

Sujet 4 : transformeurs pour données tabulaires

Comparaison de modèles de type transformeur conçus pour les données tabulaires (tableaux de lignes et de colonnes) et de méthodes classiques de référence, sur plusieurs jeux de données publics de natures différentes : taille, nombre de variables, valeurs manquantes, classes déséquilibrées. Le groupe choisit et justifie ces jeux de données en semaine 1, pour couvrir des cas variés. Ce sujet n'utilise pas de LLM.

- **Données.** Jeux de données justifiés. La même validation croisée pour tous les modèles comparés. Le traitement des valeurs manquantes et des classes déséquilibrées est décrit.
- **Observabilité.** Chaque exécution est enregistrée : hyperparamètres, temps d'entraînement, mémoire utilisée, score. Les résultats sont réunis dans un tableau reproductible. Les échecs (modèle qui ne converge pas, entraînement trop long) sont consignés comme les réussites.
- **Architecture.** Le pipeline peut être relancé à l'identique : jeux de données, hyperparamètres et code sont versionnés. Le temps d'entraînement et la mémoire sont mesurés pour chaque famille de modèles, et l'écart de coût entre transformeurs et méthodes classiques est discuté.
- **Livrables.** Pipeline de benchmark et code, résultats bruts et analyse, rapport comparatif (performance, temps de calcul, facilité de mise en œuvre, robustesse aux valeurs manquantes), deux notes techniques.

VERSION DE TRAVAIL

- **Limites.** Si une famille de modèles est meilleure sur les jeux retenus, cela ne prouve pas qu'elle le soit sur d'autres données tabulaires. Le temps de calcul disponible limite la taille des jeux de données et le nombre d'essais d'hyperparamètres.
- **Pistes techniques.** Transformeurs : TabPFN (licence de la version utilisée à vérifier : les versions récentes sont non commerciales par défaut), FT-Transformer via rtdl. Méthodes classiques : XGBoost, LightGBM. Suivi des expériences : MLflow, ou Langfuse ou Phoenix si un agent automatise une partie du pipeline via la passerelle.

Sujet 5 : « humain ou IA ? » sur un serveur Minecraft

Serveur Minecraft personnalisé où des joueurs humains et des agents fondés sur un LLM jouent ensemble. Les joueurs accomplissent des tâches définies par le groupe (collecter, fabriquer, construire, livrer des ressources) et peuvent échanger entre eux par messages et par troc. À des moments prédéfinis, chaque participant, humain ou agent, indique qui parmi les autres est humain et qui est un agent. Le projet mesure à quel point les agents passent pour des humains, et à quel point les participants savent les distinguer. Le groupe fixe en semaine 1 les tâches, les règles d'échange et la fréquence de ces désignations, puis les fait valider par les enseignants.

Contraintes sans dérogation : les participants humains sont des volontaires informés du principe du jeu ; leur identité est remplacée par un pseudonyme dans les données ; la nature de chaque joueur (humain ou agent) n'est révélée qu'après la partie ; les conditions d'utilisation de Minecraft (serveurs, comptes des joueurs) sont vérifiées dès la semaine 1.

- **Données.** Tâches et règles de jeu documentées. Corpus de parties : actions, messages, échanges, désignations de chaque participant à chaque moment, nature réelle de chaque joueur. Le nombre de volontaires et de parties est justifié par l'analyse statistique visée. L'analyse porte sur le taux de détection par participant et par type de joueur, et sur les indices utilisés (style des messages, délais de réponse, efficacité, erreurs).
- **Observabilité.** Chaque partie est enregistrée en entier (événements du serveur, messages, échanges, désignations) pour pouvoir être rejouée. Chaque appel au modèle est enregistré (contenu, jetons, temps de réponse) et relié à l'action ou au message qu'il a produit dans le jeu. Le coût et le temps de réponse sont mesurés pour chaque agent.
- **Architecture.** Le serveur de jeu (règles, tâches, échanges, désignations), les agents (perception, décision, communication) et l'enregistrement des données sont des composants séparés. Les agents utilisent les mêmes canaux que les humains (client de jeu, messagerie) et n'ont aucun accès privilégié au serveur, sinon la comparaison est faussée. Un mode de parties automatiques, sans humains, sert aux tests.

VERSION DE TRAVAIL

- **Livrables.** Serveur personnalisé et son code, agents et leur configuration, protocole de parties, données pseudonymisées, analyse des désignations, deux notes techniques.
- **Limites.** Le nombre de volontaires et de parties limite la fiabilité statistique. La latence et le coût des appels au modèle distinguent les agents des humains, quelle que soit la qualité de leur jeu. Les participants humains adaptent leur stratégie au fil des parties. Les résultats valent pour les tâches et les modèles testés.
- **Pistes techniques.** Serveur : PaperMC avec un plugin pour les tâches, les échanges et les désignations. Agents : Mineflayer (bots en Node.js), Mindcraft ou Voyager comme points de départ, ou agents construits sur la passerelle. Suivi des appels au modèle : Langfuse, Phoenix. Stockage des parties : SQLite, PostgreSQL.

Sujet 6 : monitoring d'un JupyterHub et prédiction des surcharges, crashs et redémarrages

Mise en place d'une chaîne de supervision d'un JupyterHub (serveur qui donne à plusieurs utilisateurs un accès à des notebooks Jupyter), puis construction de modèles qui prédisent les surcharges, les crashs et les redémarrages du serveur. Le groupe collecte les métriques du hub et des serveurs utilisateurs (CPU, mémoire, sessions actives, démarrages et arrêts de serveurs, erreurs, temps de réponse), les stocke sous forme de séries temporelles et les affiche dans des tableaux de bord. Les modèles prédictifs (prévision de charge, détection d'anomalies, classification d'événements) sont comparés à une référence simple, par exemple une alerte quand la mémoire ou le CPU dépasse un seuil fixe. Le groupe choisit en semaine 1 les événements à prédire, l'horizon de prédiction (combien de temps à l'avance) et les critères de succès, puis les fait valider par les enseignants.

Contraintes sans dérogation : les métriques sont collectées sans aucun contenu utilisateur (notebooks, fichiers) et les identifiants sont remplacés par des pseudonymes dès la collecte. L'accès aux métriques d'un hub en production est convenu avec ses administrateurs ; à défaut, le groupe déploie une instance de test alimentée par une charge simulée.

- **Données.** Séries temporelles du hub et des serveurs utilisateurs, avec un schéma explicite (métriques, fréquence de mesure, durée de conservation). Journal des incidents réels, qui sert de référence pour juger les prédictions, complété par des incidents provoqués sur une instance de test. Contrôle de qualité : trous de collecte, horloges décalées. Les incidents étant rares par rapport aux périodes normales, ce déséquilibre est traité et documenté.
- **Observabilité.** Tableaux de bord sur l'état du hub et sur le bon fonctionnement de la collecte elle-même. Chaque prédiction est enregistrée (horizon, niveau de confiance) puis comparée à ce qui s'est réellement produit : précision, rappel, délai d'alerte, taux de fausses

VERSION DE TRAVAIL

alertes. L'évaluation respecte l'ordre chronologique : les données de test sont postérieures aux données d'apprentissage.

- **Architecture.** Collecte, stockage, modèles et alertes sont des composants séparés, dans des conteneurs. La collecte alourdit peu le hub supervisé. Le pipeline d'entraînement peut être relancé à l'identique : données, paramètres et code sont versionnés.
- **Livrables.** Chaîne de supervision et tableaux de bord, jeu de données constitué et sa documentation, modèles prédictifs et leur comparaison à la référence, deux notes techniques.
- **Limites.** Les incidents graves sont rares : leur petit nombre limite la fiabilité statistique des modèles. Les résultats valent pour la configuration et la période observées (nombre d'utilisateurs, calendrier des cours et des examens) et se généralisent mal à un autre hub. Une alerte trop tardive ou trop fréquente est inutilisable, même avec un bon score.
- **Pistes techniques.** Collecte : métriques au format Prometheus exposées par JupyterHub et collectées par Prometheus. Affichage et alertes : Grafana. Stockage : Prometheus, TimescaleDB. Modèles : scikit-learn (détection d'anomalies, classification), statsmodels (prévision), XGBoost, LightGBM. Suivi des expériences : MLflow.

Document généré avec Quarto et le template Lapis.

VERSION DE TRAVAIL